



CENTRO UNIVERSITÁRIO DE BRASÍLIA - CEUB
CIÊNCIA DA COMPUTAÇÃO

LUCAS GONÇALVES BALDUINO - 22409139

INTELIGÊNCIA ARTIFICIAL- LABORATÓRIO 2

BRASÍLIA

2026

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.cluster import KMeans, DBSCAN


# 1. Geração dos 3 Conjuntos de Dados (Situações)

n_amostras = 500


# Dataset 1: Blobs (Agrupamentos globulares/esféricos)

X_blobs, _ = datasets.make_blobs(n_samples=n_amostras, centers=3,
                                  cluster_std=0.6, random_state=42)


# Dataset 2: Moons (Formato de duas luas entrelaçadas)

X_moons, _ = datasets.make_moons(n_samples=n_amostras, noise=0.05,
                                  random_state=42)


# Dataset 3: Circles (Círculos concêntricos)

X_circles, _ = datasets.make_circles(n_samples=n_amostras,
                                     factor=0.5, noise=0.05, random_state=42)


datasets_dict = {

    "Blobs (Grupos Esféricos)": X_blobs,

    "Moons (Luas Entrelaçadas)": X_moons,

    "Circles (Círculos Concêntricos)": X_circles

}


# 2. Definição dos Hiperparâmetros

# Para o K-means, o hiperparâmetro principal é o n_clusters (K)

kmeans_params = [3, 2, 2]
```

```
# Para o DBSCAN, os hiperparâmetros são eps (raio de vizinhança) e
min_samples (mínimo de pontos)
```

```
dbscan_params = [
    {'eps': 0.5, 'min_samples': 5}, # Para Blobs
    {'eps': 0.2, 'min_samples': 5}, # Para Moons
    {'eps': 0.2, 'min_samples': 5}  # Para Circles
]
```

```
# 3. Execução e Plotagem
```

```
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
```

```
plt.subplots_adjust(hspace=0.4)
```

```
for i, (nome_dataset, X) in enumerate(datasets_dict.items()):
```

```
    # Aplicação do K-means
```

```
    kmeans = KMeans(n_clusters=kmeans_params[i], random_state=42,
n_init=10)
```

```
    y_kmeans = kmeans.fit_predict(X)
```

```
    # Aplicação do DBSCAN
```

```
    dbscan = DBSCAN(eps=dbscan_params[i]['eps'],
min_samples=dbscan_params[i]['min_samples'])
```

```
    y_dbscan = dbscan.fit_predict(X)
```

```
    # Plot K-means
```

```
    axes[i, 0].scatter(X[:, 0], X[:, 1], c=y_kmeans,
cmap='viridis', s=30)
```

```
    axes[i, 0].set_title(f'K-means -
{nome_dataset}\n(K={kmeans_params[i]})')
```

```
    # Plot DBSCAN
```

```

# Pontos com label -1 são considerados "ruído" (outliers) pelo
DBSCAN

axes[i, 1].scatter(X[:, 0], X[:, 1], c=y_dbscan,
cmap='plasma', s=30)

axes[i, 1].set_title(f'DBSCAN -
{nome_dataset}\n(eps={dbscan_params[i]["eps"]},
min_samples={dbscan_params[i]["min_samples"]})')

plt.show()

```

1 INTRODUÇÃO

Este relatório apresenta a implementação e comparação de dois algoritmos distintos de agrupamento não-supervisionado: o K-means (baseado em centróides e distâncias geométricas) e o DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*, baseado em densidade). O objetivo é avaliar a performance e o comportamento de ambos os métodos quando expostos a conjuntos de dados com características topológicas fundamentalmente diferentes, alterando seus hiperparâmetros para buscar o agrupamento ideal.

2 METODOLOGIA E CONJUNTOS DE DADOS

Para testar a capacidade de generalização e os limites geométricos dos algoritmos, foram gerados três conjuntos bidimensionais distintos de dados artificiais utilizando a biblioteca [scikit-learn](#):

- **Situação 1 - *Blobs***: Dados distribuídos em três grupos globulares (esféricos) isolados.
- **Situação 2 - *Moons***: Dados dispostos em formato de duas meias-luas não convexas e entrelaçadas.
- **Situação 3 - *Circles***: Dados formando dois círculos concêntricos (um anel externo e um núcleo interno).

3 AJUSTE DE HIPERPARÂMETROS

Cada algoritmo exige a configuração de hiperparâmetros específicos, que foram testados empiricamente:

3.1 Hiperparâmetros do K-means:

O parâmetro principal é o K (`n_clusters`), que define a quantidade exata de grupos.

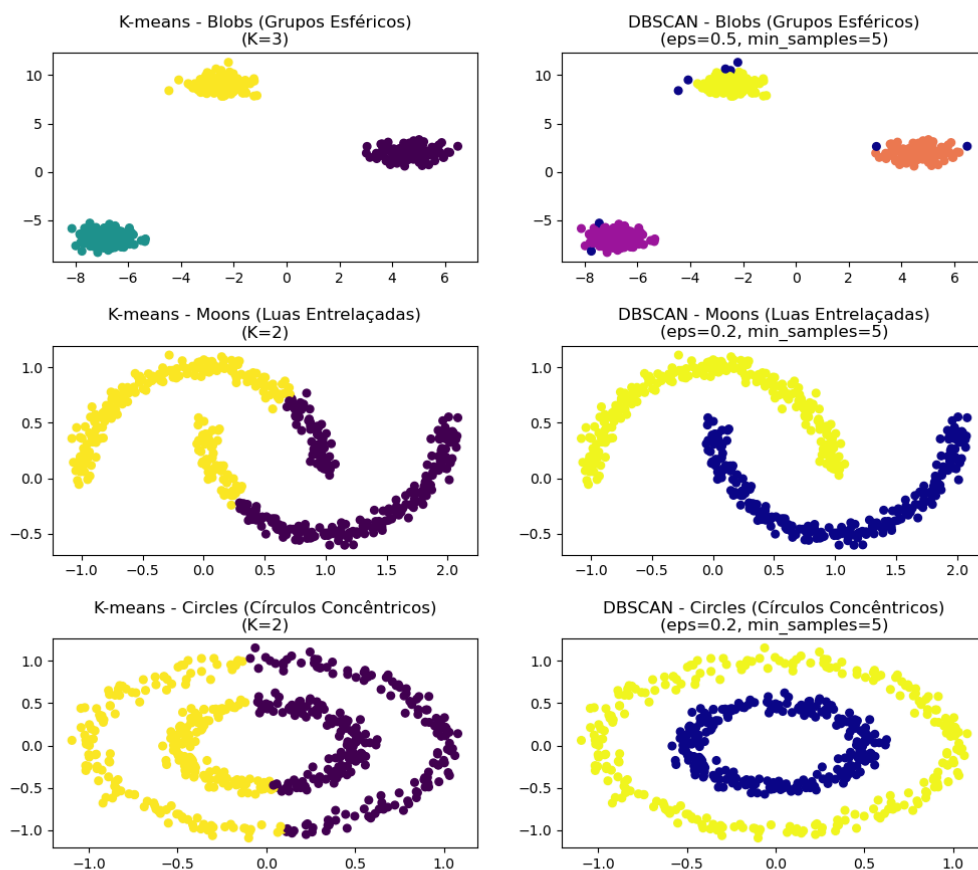
- *Blobs*: \$K = 3\$.
- *Moons*: \$K = 2\$.
- *Circles*: \$K = 2\$.

3.2 Hiperparâmetros do DBSCAN:

O DBSCAN não exige a definição prévia da quantidade de grupos, mas requer dois parâmetros fundamentais: **eps** (tamanho do raio da vizinhança) e **min_samples** (quantidade mínima de pontos dentro do raio para formar uma região densa).

- *Blobs*: **eps** = 0.5, **min_samples** = 5.
- *Moons*: **eps** = 0.2, **min_samples** = 5.
- *Circles*: **eps** = 0.2, **min_samples** = 5.

4 ANÁLISE DE RESULTADOS E COMPARAÇÃO



A execução do código revelou contrastes severos no comportamento dos algoritmos:

a) Análise do Dataset *Blobs*:

- **K-means:** Obteve sucesso absoluto. Como o algoritmo busca minimizar a variância intra-cluster e traça fronteiras lineares (diagramas de Voronoi), ele separou perfeitamente os três grupos.
- **DBSCAN:** Também obteve sucesso em identificar as três massas densas, isolando corretamente os grupos.

b) Análise do Dataset *Moons* (Meias-luas):

- **K-means:** Falhou severamente. O algoritmo dividiu os dados ao meio com um corte linear reto. Como ele agrupa pontos com base na distância até um centro fixo, ele é matematicamente incapaz de acompanhar curvaturas prolongadas.
- **DBSCAN:** Obteve sucesso absoluto. Como a métrica avalia a densidade contínua, ele rastreou o "caminho" dos pontos, separando perfeitamente a lua superior da lua inferior, sem forçar divisões lineares.

c) Análise do Dataset *Circles* (Círculos Concêntricos):

- **K-means:** Falhou novamente. O centroide de ambos os círculos fica essencialmente no mesmo lugar (o meio do anel interno). Consequentemente, o K-means cortou o círculo no meio como uma pizza, misturando o anel interno e externo em ambos os grupos.
- **DBSCAN:** Obteve sucesso. A região "vazia" entre o anel externo e o anel interno fez com que o algoritmo compreendesse que se tratam de duas densidades isoladas, separando perfeitamente o círculo de fora do círculo de dentro.

5 CONCLUSÃO

Os testes comprovam que não existe um "algoritmo perfeito" no aprendizado não-supervisionado.

O **K-means** é altamente eficiente e rápido, sendo a escolha ideal para dados que possuem distribuições regulares, convexas e esféricas. Contudo, sua limitação reside na dependência rigorosa do parâmetro k e na falha ao interpretar geometrias complexas.

O **DBSCAN** demonstrou superioridade tática ao lidar com dados de formatos arbitrários e não convexos (como *Moons* e *Circles*). Sua capacidade de delinear clusters baseando-se na continuidade da densidade permitiu extrair as estruturas corretas sem a necessidade de informar o número de grupos previamente. Além disso, a configuração do parâmetro **eps** exigiu experimentação cuidadosa, pois valores muito baixos classificam os dados como ruído (outliers), e valores muito altos fundem todos os dados em um único cluster.

REFERÊNCIAS

IBM. **O que é aprendizado não supervisionado?** [S. l.], [202-?]. Disponível em: <https://www.ibm.com/br-pt/think/topics/unsupervised-learning>. Acesso em: 20 mai. 2026.

PEDREGOSA, F. *et al.* Scikit-learn: Machine learning in Python. **Journal of Machine Learning Research**, [S. l.], v. 12, p. 2825-2830, 2011. Disponível em: <https://scikit-learn.org>. Acesso em: 20 mai. 2026.

GOOGLE. **Gemini**. Versão de maio de 2026. Mountain View: Google, 2026. Disponível em: <https://gemini.google.com>. Acesso em: 20 mai. 2026. Inteligência artificial utilizada como assistente no desenvolvimento do código em linguagem Python e responsável pela estruturação, revisão e formatação do relatório final de acordo com as normas técnicas vigentes.